CSCE 614: Cache Replacement Policy Competition/Paper Implementation

Jacob Barker, Wendi Yu, Sheena Selvakumar
CSCE, Texas A&M University
Texas A&M University
GitHub Repository: https://github.com/CSCE-614-EJKIM-F25/Term-Project-05—Repository

***Abstract*—This report evaluates two modern cache replacement policies, PACIPV and EAF, and compares them with LRU, LFU and SRRIP on the L3 cache. PACIPV uses simple insertion and promotion vectors to improve performance on instruction heavy workloads with very low hardware cost. EAF predicts block reuse by tracking recently evicted addresses with a Bloom filter structure and reduces both cache pollution and thrashing. We implement both policies and test them on PARSEC and SPEC CPU2006 benchmarks. The results show the performance and miss rate tradeoffs of each method and highlight the strengths of lightweight vector based design and reuse based prediction in practical L3 cache replacement. Among all the benchmarks, PACIPV get the best result on SPEC Integer.**

I. Introduction

Cache replacement policies are essential for determining which cache block should be evicted when space is needed. Their behavior has a strong impact on overall system performance, especially for last level caches where miss penalties are high. As modern processors execute increasingly complex workloads, traditional policies such as LRU often struggle under instruction heavy or highly irregular access patterns.

In this project, we investigate two modern replacement techniques designed to address these challenges. The first technique is PACIPV, a lightweight RRIP based mechanism that uses insertion and promotion vectors to guide cache management. It provides simple hardware requirements and is effective for instruction heavy workloads. The second technique is the Evicted Address Filter, which predicts block reuse by tracking recently evicted addresses with a Bloom filter. This prediction reduces cache pollution and mitigates thrashing by controlling how many blocks are treated as likely to be reused.

We also compare these methods against four classic baselines: Least Recently Used (LRU), Least Frequently Used (LFU), Static Re reference Interval Prediction (SRRIP), and Random replacement. LRU evicts the block that has not been accessed for the longest time, assuming temporal locality in program behavior. It is simple and performs well when locality is strong, but it degrades under cyclic scans or other patterns that cause frequent thrashing. LFU evicts the block with the lowest access count and aims to preserve blocks with long term reuse. Although LFU can outperform LRU when frequency is more stable than recency, it adapts slowly and may retain blocks that are no longer useful unless aging mechanisms are introduced. SRRIP predicts re reference intervals using a small saturating counter for each block. Newly inserted blocks receive moderate protection and recently accessed blocks are marked as likely to be reused soon. SRRIP avoids LRU's sensitivity to scans and LFU's slow adaptation while keeping overhead low. The Random policy simply selects a random block for eviction and serves as a simple baseline for comparison.

Our goal is to evaluate these modern and classic policies across benchmarks. By implementing and testing PACIPV and EAF, we aim to understand their strengths, limitations, and practical benefits for improving L3 cache performance.

Computer Science and Engineering, Texas A&M University

II. Problem description

For modern day computers, the bottleneck for certain tasks is not how fast the CPU can do computations, but rather how fast it can grab data to do work with. In other words, many tasks are now memory bound. To alleviate this problem, there needs to be a cost-effective and low latency way to bring data in to be used for the CPU; this is what the cache is for. By making hierarchical memory computers can achieve lower memory access times, while also keeping cost down for the system. with limited cache size and the CPU needing more memory than what the cache can store, we need a way to throw out old cache blocks and bring in new ones. this is where a cache replacement policy comes in. A cache replacement policy determines which cache lines will be evicted so that new ones can be inserted into the cache. The Last Level Cache, is the last level of cache usually layer 3, and if it has a miss it will have to fetch from main memory, which increases latency drastically compared to just a Layer 2 cache miss. This is why LLC's replacement policy is so important.

Last level cache performance depends heavily on the choice of replacement policy. Common approaches each have specific weaknesses.

- LRU evicts the least recently used block, but recency becomes unreliable when accesses follow cyclic patterns or when the working set exceeds the cache.
- LFU evicts the block with the lowest access count, yet its frequency metric changes slowly. A block that was once popular may stay in the cache long after its usefulness has passed.

- SRRIP assigns coarse re reference intervals that improve stability, but its fixed insertion behavior limits its ability to react when the access stream shifts between different types of locality.

To address these limitations, EAF tracks recently evicted addresses using a compact Bloom filter, allowing reuse prediction to operate independently of cache residency. This separation helps the policy distinguish short-lived pollution from blocks that belong to a high-reuse working set during thrashing. PACIPV addresses a different limitation: the rigidity of unified replacement policies. By using distinct insertion and promotion vectors for demand and prefetch accesses, PACIPV introduces targeted control over recency updates. This enables it to better accommodate instruction-heavy workloads, where the importance of demand fetches differs from that of speculative prefetches.

III. Cache Replacement Policies Techniques

A. Short description of PACIPV technique

The PACIPV technique is similar to SRRIP-HP, but instead of using standard hard-coded rules for replacement and hit promotion, it generalizes the policy by using an IPV (Insertion Promotion Vector), which in this case is implemented as an array. This generalization allows for the replacement policy to be adapted to specific workloads for the best performance, although this requires testing to determine the optimal vector values. with the right IPV it can act as SRRIP, or be less aggressive not promoting a rrpv value to what is specified in the IPV, with a max RRPV of 3 it can be 0,1,2,3. it also handles pre-fetch and demand differently, for example more aggressive with demand and less promotions with pre-fetch.

B. Implementation details of PACIPV technique

The ZSim adaptation of PACIPV was based on the SRRIP-HP policy. There are two arrays that control replacement and hit promotion: the pre-fetch IPV and the demand IPV. Both arrays are of length 5. The first 4 elements determine the hit promotion destination (RRPV on hit), and the last entry defines the insertion RRPV value (RRPV on miss/replacement). For example, if the IPV was [0,0,0,0,2], it would act exactly like the standard SRRIP-HP policy. when updating it checks if it was a demand or prefect to determine which IPV to use. it ages the same way as SRRIP, adding 1 to all RRPV if a max_rrpv value was not found.

C. Short description of EAF technique

The Evicted Address Filter (EAF) reframes cache replacement as a closed-loop control problem rather than a static one-shot prediction. Traditional policies consider only the lines that remain in the cache. For example, LRU uses recency, LFU tracks access counts, and SRRIP assigns predicted re-reference intervals—all based on in-cache information. Once a line is evicted, these policies lose visibility and cannot tell whether the eviction was a mistake until the next miss occurs.

EAF's core idea is to keep a lightweight record of lines that were just evicted. If a later access hits this "evicted set," the system gains a precise feedback signal: the working set slightly exceeds cache capacity and the policy is repeatedly discarding data that will be referenced again almost immediately. Instead of inferring reuse from indirect indicators such as recency or access counts, EAF observes the ground truth—"this specific line was evicted and returned right away"—a fact no in-cache-only predictor can capture.

Compared to LRU and LFU, EAF is not limited to a single heuristic such as recency or frequency. Compared to SRRIP, it supplements the RRPV-based prediction with direct information obtained after eviction. This post-eviction signal allows the policy to identify lines that were evicted too early and assign them a higher insertion priority on re-access. When combined with an RRIP-style framework, EAF promotes these lines to the most protected state, while the baseline predictor continues to manage all others. This reduces thrashing, avoids unstable replacement patterns, and enables the cache to correct recent replacement errors—something conventional policies cannot do.

Furthermore, the Bloom-filter implementation introduces an implicit regulation mechanism: periodic saturation and clearing naturally limit the number of lines admitted as high-reuse, effectively controlling thrashing even when the working set exceeds cache size. This coupling of direct feedback and implicit rate control is what makes EAF both robust and lightweight.

D. Implementation details of EAF technique

EAF treats replacement as a closed-loop control problem. On eviction, demand requests are inserted into the Bloom filter (filterBlocks=131072, bloomAlpha=8). On a subsequent miss, if the filter test returns true, the line is treated as a reuse candidate and inserted with RRPV=0, bypassing the normal Bimodal Insertion Policy that would assign low priority to most new lines.

Integration with RRIP: EAF supplements RRPV-based prediction with post-eviction information. The filter identifies lines evicted too early and assigns higher insertion priority on re-access, promoting them to the most protected state (RRPV=0) to reduce thrashing and correct recent replacement errors.

Implicit Regulation: The Bloom filter provides implicit rate control: periodic saturation and clearing naturally limit the number of lines admitted as high-reuse, controlling thrashing even when the working set exceeds cache size. When the filter reaches capacity, it clears all entries, resetting the prediction state.

Set Dueling: EAF uses set dueling (leaderSets=32, pselBits=10) to dynamically choose between EAF-enabled and baseline SRRIP policies. Leader sets are assigned using a prime-number stride, with half using EAF and half using baseline. The policy selector (psel) trains on every replacement: when an EAF leader set has a miss, psel decrements; when a baseline leader set has a miss, psel increments. Non-leader sets follow the majority vote (psel.cur() ¿= psel.mid()), enabling workload-adaptive policy selection.

IV. Evaluation Methods

To evaluate each policy we used the provide benchmarks. We ran these benchmarks in zsim on the TAMU linux 2 server. We created a script that would use the HW4 run script to run each benchmark for each replacement policy this way we could automate it, or else it could have taken a extremely long time to manually run each benchmark.

After we gathered all of the output files from the benchmarks for each replacement policy, we feed it into a python program that would display all of the policy performance against each other. These resulting plots are the main tools we used to evaluate the polices performance. We made 9 plots in total, for 3 benchmark types(PARSEC, SPEC 2006 INT, SPEC 2006 FP) and for each of those a 3 graphs for total cycles, IPC, and MPKI.

To determine if a replacement policy performed well we looked at each benchmark and compared it with the others for that same benchmarks. we did this for every benchmark that showed a significant difference between replacement policy used. If the IPC, MPKI, and cycles were almost the same no matter the benchmark we concluded replacement policy did not matter. This was made evident by the addition of the Rand replacement policy. If all other policy did the same as this one, it was clear that the polices did not effect miss-rate in any examinable way.

V. Results and Evaluation

A. Benchmarks

We use ZSim, a user-level x86 simulator, to evaluate different cache replacement techniques. Two standard benchmark suites are used in this study:

- **SPEC CPU2006 (single-threaded):** *bzip2, gcc, mcf, hmmer, sjeng, libquantum, cactusADM, namd, soplex, calculix, lbm*
- **PARSEC (multi-threaded):** *blackscholes, bodytrack, canneal, fluidanimate, freqmine, streamcluster, swaptions, x264*

we could not get (xalancbmk, milc, leslie3d) to run.

B. System Configuration

We compare various cache replacement algorithms for a 8-core configuration with a 16 MB shared last-level cache.

We evaluate the following replacement policies:

- LRU
- LFU
- SRRIP
- Random
- PACIPV
- EAF

PACIPV_paper used the IPV recommended in the paper, demand IPV [0,1,1,0,3] pre-fetch IPV[0,1,0,0,3]
Note that PACIPV is a generalization of SRRIP, so any results SRRIP got PACIPV could have got too.

Random was added because it tells us if other policy were useful or not. If it did the same as random then likely there was not pattern it could take advantage of, it did better then there was some locality to take advantage of, and if it did worse it meant the benchmark was likely thrashing removing data right before it was going to be used, and that there was no pattern to take advantage of.

C. Simulation Flow

For each benchmark and each replacement policy, we perform:

1) Configure simulation with selected replacement policy.
2) Run benchmark under ZSim.
3) Collect execution statistics from simulator output:
   - Total cycles
   - Total instructions
   - LLC misses

D. Metrics

Cycle graphs: The cycle graphs are useful for seeing how long each program ran for, the more cycles the longer it takes for the program to finish. Lower is better.

$$total\_cycles = cycles + cCycles \tag{1}$$

IPC graphs: These are useful for seeing how many instructions could be executed every cycle. Higher is better.

$$IPC = total\_instruction/total\_cycles \tag{2}$$

MPKI graphs: Misses per 1000 instruction, is a indicator of how good a replacement policy is at choosing the right cache line to evict, although a lower MPKI does not always indicate a better performance or greater speed up. Lower is better.

$$total\_misses = mGETS + mGETXIM + mGETXSM \tag{3}$$

$$MPKI = (total\_misses/total\_instruction) * 1000 \tag{4}$$

E. PARSEC results

PARSEC benchmarks were used with 16MB L3 cache and used 8 cores.

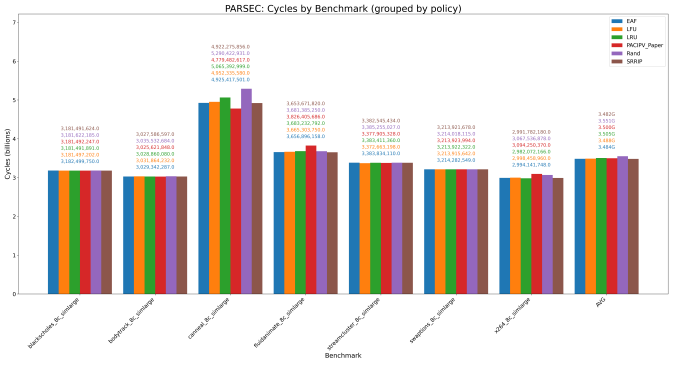

Fig. 1. PARSEC: Cycles by benchmark

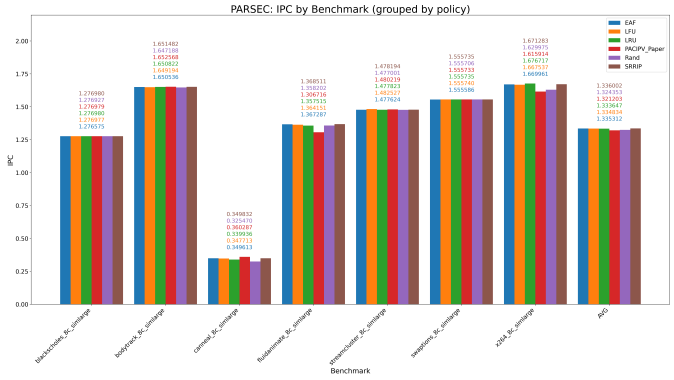

Fig. 2. PARSEC: Instructions per cycle (IPC)

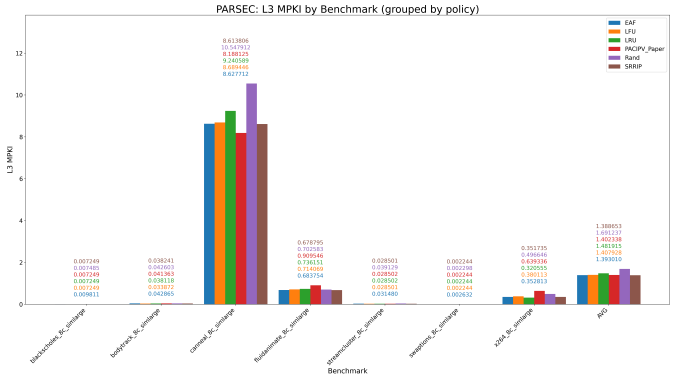

Fig. 3. PARSEC: L3 MPKI by benchmark

a) Cycles: From the results gathered Fig. 1 its clear to see that for the majority of the benchmarks the replacement policy did not effect cycle count that much. This is clear by addition of Rand, a random replacement replacement policy, and how it did just as good as the others, which should not be the case if the policy were actually being useful. However on 1 benchmark a noticeable variation was observed with canneal. For this one benchmark PACIPV did the best by 200 million cycles. This leads to the conclusion that this benchmark was likely a instruction heavy workload, however it was not only having an IPC of 0.36, which means it must be having some other interaction that the original PACIPV paper did not mention. EAF shows small gains on the PARSEC single-threaded tests, but comparable results with other baselines. Because it builds on RRIP, its results stay close to the RRIP baseline. Most PARSEC workloads do not create much thrashing or premature evictions, so the Bloom filter has few chances to detect useful reuse.

b) IPC: Shown in Fig 2 the benchmark that PACIPV finished the fastest canneal, also had the highest IPC of 0.36 while the runner up had just under 0.35. This means that PACIPV can perform on low instruction heavy workload, different from what the original paper concluded.

c) MPKI: Shown in Fig 3 benchmarks had fewer than 1 in 100000 instruction misses, meaning that likely the only misses were the first misses and that almost no evictions occurred. This is because a 16MB cache could likely hold all the data without needing to evict. Out of the useful benchmarks (canneal, fluidanimation, and x264) PACIPV did the best for canneal and the worst for fluidanimation and x264 out of all tested replacement polices even random. This means that for canneal it was able to prevent thrashing and took advantage of the memory access structure, but for the other 2 it was likely thrashing which caused it to do the worse for those 2.
EAF shows minimal impact in the PARSEC MPKI results. Miss rates are extremely low for most benchmarks, so very few evictions occur and the Bloom filter does not have many opportunities to correct premature evictions.

F. SPEC FP results

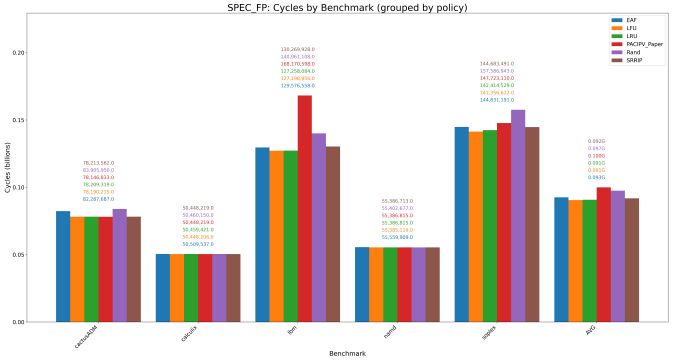

Fig. 4. SPEC FP: Cycles by benchmark

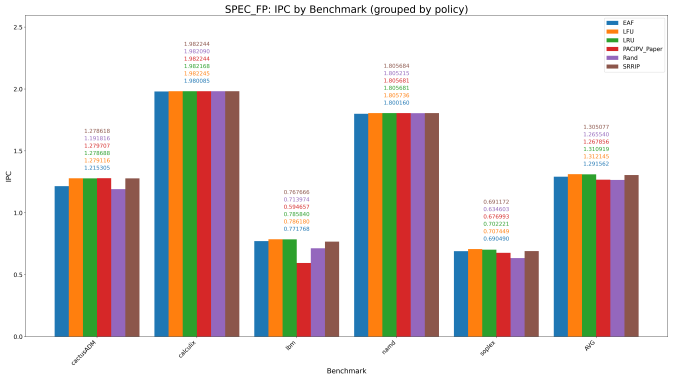

Fig. 5. SPEC FP: Instructions per cycle (IPC)

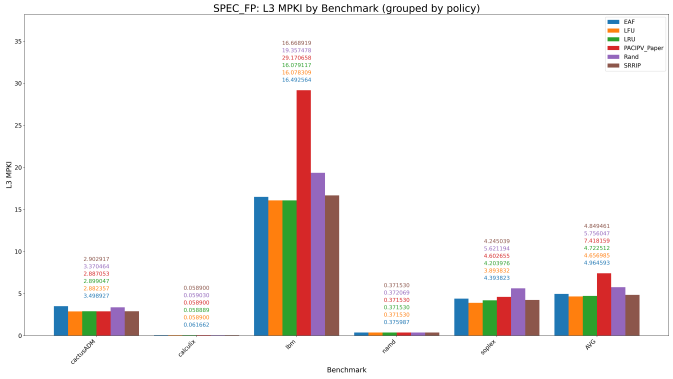

Fig. 6. SPEC FP: L3 MPKI by benchmark

a) Cycles: From the results gathered in Fig. 4 its clear that PACIPV did not preform well. on all of the benchmarks with variations these being IBM and soplex, PACIPV performed the worst(besides rand). PACIPV took around 40 million more cycles compared to LRU, making it take around 32% longer or more cycles.

b) IPC: Shown in Fig 5 is the IPC graph, its clear that SRRIP had a comparatively much lower IPC of around 0.59 to LRU which had a IPC of 0.78 for IBM benchmark. this one bench mark made PACIPV go from average to worst for SPEC FP. This is a low instruction count benchmark which is not meant for these IPV for PACIPV to do well with.

c) MPKI: Shown in Fig 6 is the MPKI for SPEC FP benchmarks. For PACIPV the only one that stands out and deserves the focus is IBM benchmark, which had double the MPKI of LRU and is about 1/3 higher then RAND. this means that it was thrashing extremely hard. The reason could be that in between the data that is usually used again in a loop it takes some special data which throws the usable data out. On a replaceable RRPV is set to 3 which means it can be evicted if its does not have a hit right away, basically it seems like if the cache is full and its fetching new data the data just grabbed is up for eviction, this would seem really bad and it is for this benchmark but not for all benchmarks as will be shown for the next benchmark section.

G. SPEC INT results

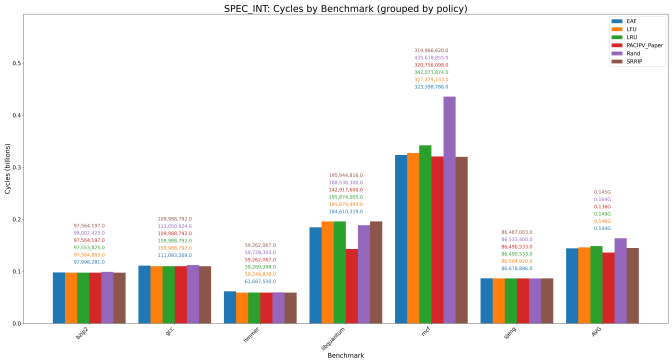

Fig. 7. SPEC INT: Cycles by benchmark

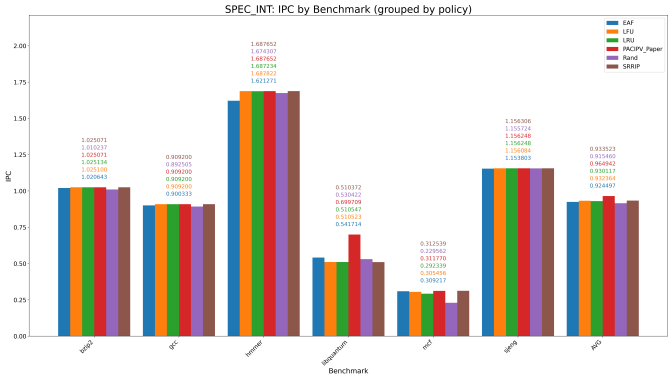

Fig. 8. SPEC INT: Instructions per cycle (IPC)

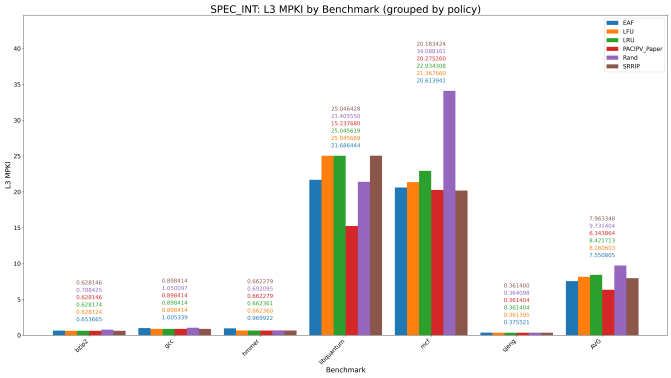

Fig. 9. SPEC INT: L3 MPKI by benchmark

a) Cycles: From the results gathered Fig. 7 it clear that for most benchmarks the policy did not effect misstate IPC or total cycles. however for the benchmark libquantum, PACIPV outperforms the others drastically. It took around 40 million fewer cycles that is about a 22 percent speed up. Overall for SPEC INT benchmark average it achieved a 6.6% speed up.

b) IPC: Shown in Fig 8 its visible that PACIPV had the highest IPC overall, specify for libquantum, however this benchmark was not instruction heavy and it still performed amazing. PACIPV with the current configuration(for instruction heavy workloads) was supposed to perform well on instruction heavy programs but it has been shown to also perform very well on more memory heavy programs, which is different from the original paper.

c) MPKI: Shown in Fig 9 PACIPV also had the lowest MPKI which makes complete sense given results from the other 2 figures. The 2 high missrate benchmarks are likly memory access heavy, this is what PACIPV seems to be excel at, but still not recorded in the original paper. EAF gets a big improvement compared with LRU, LFU and SSRIP, noticed that its based on RRIP, so EAF's bloom filter strcuture works in this high mssing rate dataset to reduce cache pollution and trashing.

H. final comparison

PACIPV shined on SPEC 2006 integers having a speed up of around 6% thanks to libquantum which had a 22% speed up compared with the other replacement policy.
for the other benchmarks PARSEC and SPEC 2006 FP it performed around the middle of the road. However since PACIPV is a generalization of SSRIP, it could also be configured and ran to perform exactly the same as SRRIP, so all the results SSRIP got PACIPV could have gotten as well. PACIPV in the original paper was stated to be for instruction heavy workloads, however in our benchmarks PACIPV did much better in Memory heavy and high miss-rate benchmarks. This discrepancy is likely a result of the small sample size. In the Paper they used over 100 benchmarks while we used fewer then 20, and only a few off those did any of the policy stand out from the random policy. This leads to the conclusion that the benchmarks sample size might be to small to accurately show each replacements policy true characteristics and average performance. The PACIPV seems highly volitive, around average for most benchmarks but for one of them yielded a 22% speed up, and for another benchmark, it took 32% more cycles to complete. this volatility might be why in the paper PACIPV got a 6.6% over LRU thanks to this volitionally, but performed around average in our benchmarks. if IBM benchmark was removed PACIPV would be the clear winner by a few percentage points just like in the report. If libquantum was removed PACIPV would have done the worst my a few percentage points.

EAF mostly can reach a comparable result with SRRIP on all the three benchmarks, so in most cases its Bloom filter and set dueling design do not hurt the base performance. In some heavy missing rate cases, such as libquantum, it has big improvement about 13% compared with SRRIP, which shows its efficacy in this case. However, in some cases that EAF does not perform well. These are primarily low-miss-rate workloads (e.g., blackscholes, swaptions, bodytrack) where the Bloom filter's false positive rate becomes the dominant factor.
Why EAF fails in low-miss-rate scenarios: In workloads with very low MPKI (0.05), the filter saturates quickly from evictions, but the actual reuse rate is too low to justify the overhead. False positives (estimated 5-10% with current configuration) cause non-reused lines to receive RRPV=0 (highest priority), displacing truly reusable lines. This pollution has a larger relative impact in low-miss-rate workloads because each miss is more costly, and the filter's benefit-to-cost ratio becomes negative. For example, on blackscholes the mssing rate is very low, almost close to 0, EAF MPKI overhead +35.34% vs SRRIP (0.0098 vs 0.0072 MPKI)

The fundamental issue: EAF assumes that the benefit of correctly predicting reuse outweighs false positive costs. In low-miss-rate workloads, this assumption breaks down because (1) fewer true reuses exist to predict, (2) false positives have proportionally larger impact, and (3) the filter saturates faster relative to the workload's eviction pattern. The lack of confidence mechanisms prevents EAF from gracefully degrading in these scenarios.

ACKNOWLEDGMENT

No communication with anyone outside of our group was done for this project.

CONCLUSIONS

PACIPV was an improvement compared to the other policy for SPEC 2006 INT, thanks to a single benchmark having around a 22% speed up. For SPEC FP it was the opposite having 1 benchmark where it was around 32% slower. For PARSEC it was not remarkable or tearable but performed around average. From these results PACIPV shown to be extremely volatile, if libquantum was removed PACIPV would have performed the worst by a few percentage points, the opposite holds true for removing the IBM benchmark. This observed volatility makes PACIPV at least for the configuration used in the paper a risky choice and needs to be tested further to get a more acuate indicator of its success.

EAF performs well in high-miss-rate scenarios such as libquantum, where its reuse prediction is highly effective. In contrast, it provides limited benefit on low-miss-rate workloads, where false positives in the Bloom filter become more common. Overall, EAF delivers performance that is comparable to its baseline RRIP variant, SSRIP. These results indicate that in high-intensity, high-miss-rate environments, augmenting a baseline replacement policy with EAF's Bloom-filter feedback can provide meaningful improvements—conditions that align with many modern workloads.

The best replacement policy is highly situational and is highly depend on the program being run, memory being accessed, cache size, set associativity, and ways of connectivity. For a replacement policy to perform well it would need to

take advantage of the benchmarks memory access patterns and prevent thrashing.

REFERENCES

[1] ”High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP)”, Aamer Jaleel, Kevin Theobald, Simon C. Steely Jr, and Joel Emer. In International Sympo- sium on Computer Architecture (ISCA), Saint-Malo, France, June 2010.

[2] Saba Mostofi, Setu Gupta, Ahmad Hassani, Krishnam Tibrewala, Elvira Teran, Paul V. Gratz, and Daniel A. Jiménez. 2025. Light-weight Cache Replacement for Instruction Heavy Workloads. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA ’25). Association for Computing Machinery, New York, NY, USA, 1005–1019. https://doi.org/10.1145/3695053.3730993

[3] V. Balaji, N. Crago, A. Jaleel, and B. Lucia, ”P-opt: Practical optimal cache replacement for graph analytics,” in 2021 IEEE International Symposium on High Performance Computer Architecture (HPCA), IEEE, 2021.

[4] Seshadri, Vivek, et al. ”The evicted-address filter: A unified mechanism to address both cache pollution and thrashing.” Proceedings of the 21st international conference on Parallel architectures and compilation techniques. 2012.